SecurePulse Security Operations Platform — Official Document

SecurePulse Mobile — Media Placement Audit 🎬

Audit Property	Value
Audit Date	2026-09-01
Documents Scanned	PROJECT_REPORT.md, WALKTHROUGH.md, ROADMAP.md, MINDMAP.md, DOCUMENTATION_STATUS.md
Total Media Items Recommended	**70**
Screenshots	41
Videos	13
Diagrams (exported PNG)	5
Code Evidence / API Logs	11
Rule	No fake media. All items are PENDING capture. Filenames are suggestions only.

Priority Matrix

Level	Criteria
CRITICAL	Cited in List of Figures, abstract, results, or directly fills a named placeholder. Missing item invalidates a claim.
HIGH	Supports a major feature chapter or demo workflow. Expected in a professional technical report.
MEDIUM	Adds depth and professionalism. Document is readable without it but weaker.

Part 1 — PROJECT_REPORT.md

Chapter 1 — Introduction

ID	Priority	Evidence Type	Description	Suggested Filename	Why Useful
M-001	MEDIUM	SCREENSHOT	SecurePulse live dashboard on a physical Android device — hero opening image	`fig_1_1_product_hero_android.png`	Creates compelling first impression. Confirms product is real, not theoretical.
M-002	MEDIUM	DIAGRAM	Context diagram: SOC analyst receiving a Wazuh alert on mobile	`fig_1_2_context_overview.png`	Grounds reader in use case before technical detail begins.

Chapter 2 — Related Technologies

ID	Priority	Evidence Type	Description	Suggested Filename	Why Useful
M-003	MEDIUM	SCREENSHOT	Wazuh Manager web dashboard showing agent-generated alerts	`fig_2_1_wazuh_dashboard_reference.png`	Illustrates upstream data source SecurePulse consumes. Grounds Section 2.3.
M-004	MEDIUM	SCREENSHOT	GitHub repository Security tab with Dependabot / code scanning results	`fig_2_2_github_security_tab.png`	Shows GitHub security pipeline SecurePulse integrates with (Section 2.4).
M-005	MEDIUM	SCREENSHOT	Semgrep CLI terminal output or Semgrep App findings page	`fig_2_3_semgrep_findings_reference.png`	Shows raw tool output that SecurePulse processes and surfaces to mobile.

Chapter 4 — System Architecture

ID	Priority	Evidence Type	Description	Suggested Filename	Why Useful
M-006	CRITICAL	DIAGRAM	Full SecurePulse end-to-end system topology: Mobile → FastAPI → PostgreSQL → Wazuh/GitHub/Semgrep	`fig_4_1_overall_architecture.png`	LISTED AS FIGURE 4.1 in the report. Most important single figure. Completely missing.
M-007	CRITICAL	DIAGRAM	Mobile Clean Architecture layers: Presentation, State, Domain, Data with Riverpod provider tree	`fig_4_2_mobile_clean_architecture.png`	LISTED AS FIGURE 4.2. Entire mobile architecture chapter references this.
M-008	HIGH	DIAGRAM	FastAPI async backend component pipeline: Routers, Services, Repositories, DB, WebSocket hub	`fig_4_3_backend_architecture.png`	LISTED AS FIGURE 4.3. Validates Section 4.4 backend architecture claims.
M-009	HIGH	SCREENSHOT	Architecture poster annotated screenshot — the ARCHITECTURE SCREENSHOT placeholder at line 619	`fig_4_4_architecture_overview_poster.png`	Makes mermaid diagrams portable as static images in PDF.
M-010	HIGH	SCREENSHOT	Annotated 5-tab shell navigation hierarchy screenshot	`fig_4_4_5tab_navigation_shell.png`	LISTED AS FIGURE 4.4. Confirms declarative shell routing claim.
M-011	CRITICAL	DIAGRAM	WebSocket connect → reconnect → HTTP polling fallback sequence diagram	`fig_4_5_websocket_fallback_flow.png`	LISTED AS FIGURE 4.5. Section 4.6 transport resilience claim depends on this.
M-012	HIGH	DIAGRAM	AI Copilot: prompt → demo branch vs. FastAPI /v1/ai/chat live branch flow	`fig_4_6_ai_copilot_flow.png`	LISTED AS FIGURE 4.6.
M-013	HIGH	SCREENSHOT	Mobile screenshot showing rendered Flutter UI alongside architecture description (placeholder line 684)	`fig_4_7_flutter_mobile_rendered.png`	Shows actual rendered mobile UI.
M-014	MEDIUM	SCREENSHOT	Backend rendered screenshot (line 731): FastAPI Uvicorn startup log or Swagger UI	`fig_4_8_backend_rendered.png`	Provides evidence that described backend components are real and running.

Chapter 5 — Development Methodology

ID	Priority	Evidence Type	Description	Suggested Filename	Why Useful
M-015	MEDIUM	SCREENSHOT	Git commit history graph (git log --oneline --graph) showing development phases	`fig_5_1_git_commit_history.png`	Traceable evidence of iterative development methodology described in Section 5.1.

Chapter 6 — System Implementation

ID	Priority	Evidence Type	Description	Suggested Filename	Why Useful
M-016	CRITICAL	SCREENSHOT	Splash screen with shimmer animation during startup (placeholder line 1191)	`fig_6_1_splash_shimmer_screen.png`	First user-facing screen. Confirms polished launch UX and app loads correctly.
M-017	CRITICAL	SCREENSHOT	Login screen with email, password, GitHub SSO button, biometric icon (placeholder line 1259)	`fig_6_2_login_screen.png`	Central authentication surface. Critical for Section 6.5.
M-018	CRITICAL	SCREENSHOT	Native Android biometric challenge modal (fingerprint or face recognition prompt)	`fig_6_3_biometric_challenge.png`	Hardware biometric auth is a documented security feature. Must be evidenced.
M-019	CRITICAL	SCREENSHOT	Repository list screen: language badges, health grades A-F, finding counts (placeholder line 1272)	`fig_6_4_repositories_list.png`	Core monitoring feature. Supports Section 6.8.
M-020	HIGH	SCREENSHOT	Repository detail: scan history, individual findings, trigger scan button	`fig_6_4b_repository_detail.png`	Validates SAST drill-down workflow.
M-021	CRITICAL	SCREENSHOT	AI Copilot conversation showing CVE-2024-3094 remediation with code block (placeholder line 1317)	`fig_6_5_ai_copilot_cve_response.png`	Specific CVE response cited in Section 6.14 and test Section 8.5.
M-022	CRITICAL	SCREENSHOT	Executive dashboard: posture gauge 88%, vulnerability donut chart, service badges (placeholder line 1331)	`fig_6_6_executive_dashboard.png`	Most-cited UI screen in the entire report.
M-023	CRITICAL	SCREENSHOT	SOC alert feed with 6 severity-tagged cards: Wazuh, Splunk, Sentinel, Semgrep, Elastic, GitHub (placeholder line 1341)	`fig_6_7_soc_alerts_feed.png`	Alert triage interface is a primary deliverable. Section 6.16.
M-024	HIGH	SCREENSHOT	Alert detail: origin IP, syslog text, remediation recommendation, Quarantine/Acknowledge buttons	`fig_6_7b_alert_detail_triage.png`	Validates analyst triage workflow.
M-025	HIGH	SCREENSHOT	Settings screen: environment switcher expanded, demo mode toggle (placeholder line 1356)	`fig_6_8_settings_environment.png`	Validates Section 6.17 environment switching and diagnostics.
M-026	HIGH	SCREENSHOT	Settings → Health Ping result: Render Cloud latency and HTTP 200 status	`fig_6_8b_health_ping_result.png`	Proves live backend connectivity diagnostic works in practice.
M-027	HIGH	SCREENSHOT	Scans screen showing Semgrep scan history with severity breakdown and timestamps	`fig_6_9_scans_screen.png`	Validates SAST scan result surfacing in Section 6.12.
M-028	HIGH	SCREENSHOT	Report generation screen with compliance framework selector	`fig_6_10_report_generation.png`	Validates PDF compliance report feature.

ID	Priority	Evidence Type	Description	Suggested Filename	Why Useful
			(SOC 2, ISO 27001, PCI-DSS, HIPAA)		
M-029	HIGH	SCREENSHOT	Generated PDF compliance report open on Android: audit ID, posture score, controls table	`fig_6_10b_pdf_report_rendered.png`	Direct evidence of PDF output claimed across Sections 6 and 7.15.
M-030	HIGH	SCREENSHOT	Directory tree structure (lib/ folder expansion) confirming Clean Architecture packages (LISTED AS FIGURE 6.1)	`fig_6_11_directory_structure.png`	Cited as Figure 6.1. Supports Section 6.1 project structure narrative.

Chapter 6, Section 6.22 — Walkthrough Videos

ID	Priority	Evidence Type	Description	Suggested Filename	Why Useful
M-045	CRITICAL	VIDEO	Full app launch → biometric auth → dashboard load → posture score animation (placeholder line 1391)	`media/videos/demo_01_launch_dashboard.mp4`	Demonstrates complete launch and authentication UX.
M-046	CRITICAL	VIDEO	Live Wazuh event → FastAPI → real-time alert appearing on physical Android device (placeholder line 1398)	`media/videos/demo_02_wazuh_realtime_alert.mp4`	The flagship demonstration. Proves real-time pipeline end-to-end.
M-047	CRITICAL	VIDEO	AI Copilot: user types CVE-2024-3094 → animated response with remediation code (placeholder line 1405)	`media/videos/demo_03_ai_copilot_cve.mp4`	Demonstrates AI interaction model and response quality.

Chapter 7 — Security Design

ID	Priority	Evidence Type	Description	Suggested Filename	Why Useful
M-031	HIGH	SCREENSHOT	flutter_secure_storage key visible via Android Keystore inspection (ADB) (placeholder line 1564)	`fig_7_1_keystore_token_entry.png`	Proves JWT is in hardware-backed enclave, not SharedPreferences.
M-032	MEDIUM	SCREENSHOT	Android Studio logcat: ApiClient interceptor showing Authorization: Bearer [REDACTED]	`fig_7_2_credential_redaction_log.png`	Proves Section 7.14 credential redaction in the diagnostic logger.
M-033	MEDIUM	SCREENSHOT	Network inspection showing HTTPS TLS 1.3 handshake to Render Cloud endpoint	`fig_7_3_tls_https_inspection.png`	Validates Section 7.8 transport security claim.

Chapter 8 — Testing and Results

ID	Priority	Evidence Type	Description	Suggested Filename	Why Useful
M-034	CRITICAL	SCREENSHOT	flutter test terminal output showing 15/15 tests passed with green checkmarks (placeholder line 1752, FIGURE 8.1)	`fig_8_1_flutter_test_15_passed.png`	LISTED AS FIGURE 8.1. Primary empirical result of the project. Absolutely mandatory.
M-035	CRITICAL	SCREENSHOT	flutter analyze terminal output showing No issues found!	`fig_8_2_flutter_analyze_clean.png`	Validates Section 8.7 static analysis audit. Key quality metric.
M-036	HIGH	SCREENSHOT	API response screenshot (placeholder line 1652): login endpoint returning JWT payload	`fig_8_3_api_login_response.png`	Empirical evidence for Section 8.4 REST API contract testing.
M-037	MEDIUM	SCREENSHOT	Android device running app on physical phone confirming APK deployment and launch	`fig_8_4_physical_device_install.png`	Validates Section 8.11 platform compatibility. Confirms 63.6 MB APK installs.

Chapter 9 — Deployment and Release

ID	Priority	Evidence Type	Description	Suggested Filename	Why Useful
M-038	CRITICAL	SCREENSHOT	Render Cloud web service dashboard showing Live deployment status and memory metrics (placeholder line 1845)	`fig_9_1_render_dashboard.png`	Directly confirms cloud deployment claim.
M-039	CRITICAL	SCREENSHOT	Live /health API endpoint JSON response from secureguard-backend-7eqm.onrender.com/health (placeholder line 1826)	`fig_9_2_api_health_live.png`	Proves backend is deployed and responsive in production.
M-040	HIGH	SCREENSHOT	Render Cloud environment variables page: keys visible, values obscured	`fig_9_3_render_env_vars.png`	Shows secrets are externalized, not hardcoded. Validates Section 9.6.
M-041	CRITICAL	SCREENSHOT	flutter build apk --release terminal completion output or Android Studio APK wizard (placeholder line 1904)	`fig_9_4_android_release_build.png`	Validates Section 9.9 Android release engineering.
M-042	CRITICAL	SCREENSHOT	securepulse-release.apk in file manager showing 63.6 MB size	`fig_9_4b_apk_artifact.png`	APK file confirmed to exist at root. Simple proof-of-build.
M-043	HIGH	SCREENSHOT	Google Play Console upload screen or Play Store listing draft (placeholder line 1926)	`fig_9_5_play_store_listing.png`	Validates Section 9.11 Play Store preparation.
M-044	MEDIUM	SCREENSHOT	Render Cloud live log stream showing FastAPI startup: Uvicorn running on 0.0.0.0:8000	`fig_9_6_render_live_logs.png`	Operational evidence that deployed backend is serving requests.
M-048	CRITICAL	VIDEO	Demo Mode → Live Mode switch → health ping OK → alerts load from Render (placeholder line 1948)	`media/videos/demo_04_live_mode_switch.mp4`	Proves the Demo/Live dual-mode architecture works end-to-end.

Part 2 — WALKTHROUGH.md

ID	Priority	Section	Evidence Type	Description	Suggested Filename	Why Useful
M-049	HIGH	Sec 8: Start FastAPI	SCREENSHOT	Terminal: uvicorn main:app --reload with startup log	`fig_w_01_fastapi_startup.png`	Proves local dev server boots. Essential for technical walkthrough.
M-050	CRITICAL	Sec 9: Health Check	SCREENSHOT	Browser or curl output of GET /health returning {"status":"ok"} (line 156)	`fig_w_02_health_check_response.png`	Health check section has no evidence without this.
M-051	CRITICAL	Sec 10: Auth	SCREENSHOT	Login screen with filled email/password fields before tap (line 174)	`fig_w_03_login_screen_filled.png`	Authentication section requires visual evidence.
M-052	CRITICAL	Sec 12: Emulator	SCREENSHOT	Android Studio AVD Manager with emulator running SecurePulse	`fig_w_04_avd_emulator_running.png`	Proves emulator setup and app runs on API 34+.
M-053	CRITICAL	Sec 14: Dashboard	SCREENSHOT	Dashboard in Dark Mode: posture gauge, chart, service badges (line 227)	`fig_w_05_dashboard_dark.png`	Dashboard section anchor.
M-054	HIGH	Sec 14: Dashboard	SCREENSHOT	Dashboard in Light Mode — dual theme demonstration	`fig_w_06_dashboard_light.png`	Shows theme switching capability.
M-055	CRITICAL	Sec 15: Repos	SCREENSHOT	Repository list: all 5 demo repos with language badges and health grades (line 245)	`fig_w_07_repositories_list.png`	Required for the Repositories section.
M-056	HIGH	Sec 15: Repos	SCREENSHOT	Repository detail: scan history and findings list drill-down	`fig_w_08_repository_detail.png`	Validates SAST drill-down workflow.
M-057	CRITICAL	Sec 16: Alerts	SCREENSHOT	SOC Alert feed with severity-ordered colour-coded cards (line 264)	`fig_w_09_alerts_feed.png`	Core feature evidence.
M-058	HIGH	Sec 16: Alerts	SCREENSHOT	Alert detail: Wazuh SSH brute force raw event data and Remediation card	`fig_w_10_alert_wazuh_detail.png`	Specific Wazuh alert matching demo data cited in report.
M-059	CRITICAL	Sec 17: AI	SCREENSHOT	AI Copilot: SQL injection query with formatted response	`fig_w_11_ai_copilot_sql_chat.png`	Essential AI feature proof.

ID	Priority	Section	Evidence Type	Description	Suggested Filename	Why Useful
				and code block (line 282)		
M-060	HIGH	Sec 17: AI	SCREENSHOT	AI Copilot: secret/key rotation response with AWS Secrets Manager code sample	`fig_w_12_ai_copilot_secret.png`	Shows second distinct AI response type.
M-061	HIGH	Sec 18: Settings	SCREENSHOT	Settings: environment dropdown expanded showing all 4 URL options (line 300)	`fig_w_13_settings_env_dropdown.png`	Validates environment switcher.
M-062	HIGH	Sec 21: WebSocket	SCREENSHOT	Android Studio logcat: [WebSocketService] Status: connected log line	`fig_w_14_websocket_connected_log.png`	Log evidence of live WebSocket connection event.
M-063	CRITICAL	Sec 21: WebSocket	VIDEO	WebSocket connect → live alert from Render → appearing in UI within seconds (line 340)	`media/videos/demo_05_websocket_live.mp4`	Demonstrates live streaming pipeline.
M-064	HIGH	Sec 22: Wazuh	SCREENSHOT	Wazuh Manager web UI: agent list and generated alert	`fig_w_15_wazuh_agent_alerts.png`	Links upstream Wazuh data to SecurePulse alert feed.
M-065	HIGH	Sec 23: GitHub	SCREENSHOT	GitHub repository webhook delivery log with push event payload	`fig_w_16_github_webhook_delivery.png`	Connects GitHub integration to mobile monitoring feature.
M-066	HIGH	Sec 24: Semgrep	SCREENSHOT	Semgrep SAST findings screen in SecurePulse with severity filter (line 393)	`fig_w_17_semgrep_findings_mobile.png`	Validates Semgrep result surfacing in app.
M-067	CRITICAL	Sec 25: Cloud	SCREENSHOT	Render dashboard: SecurePulse backend service active and healthy (line 435)	`fig_w_18_render_service_live.png`	Cloud deployment walkthrough anchor.
M-068	CRITICAL	Sec 30: Final	VIDEO	End-to-end: Demo Mode → Live Mode → WebSocket → alert → AI remediation (line 473)	`media/videos/demo_06_e2e_lifecycle.mp4`	Capstone walkthrough demonstration.

Part 3 — Supporting Documents

ROADMAP.md

ID	Priority	Section	Evidence Type	Description	Suggested Filename	Why Useful
M-069	MEDIUM	Phase Overview	DIAGRAM	Gantt or swimlane rendering of the 17-phase roadmap timeline	`fig_roadmap_gantt.png`	Transforms text table into visual timeline. Communicates project lifecycle at a glance.

DOCUMENTATION_STATUS.md

ID	Priority	Section	Evidence Type	Description	Suggested Filename	Why Useful
M-070	MEDIUM	Sec 4: Features	SCREENSHOT	flutter test 15/15 result cross-referenced from M-034	(shared with M-034)	Audit status document gains credibility with embedded test pass screenshot.

Part 4 — Video Workflow Scripts (Priority Order)

V-01 — CRITICAL: Real-Time Wazuh → SecurePulse → Phone Alert

File: `media/videos/demo_02_wazuh_realtime_alert.mp4`

Steps to record:
1. Wazuh Manager visible — trigger a synthetic SSH brute-force alert
2. FastAPI backend log — show WebSocket broadcast or HTTP poll dispatch
3. Android phone screen record — alert card appears in SOC feed within seconds
4. Tap the alert — show full triage detail view

Why critical: This is the primary technical claim of the entire project. No other evidence proves it as convincingly.

V-02 — CRITICAL: GitHub Webhook → SecurePulse Mobile

File: `media/videos/demo_07_github_webhook.mp4`

Steps to record:
1. GitHub repository — push a commit or open a PR
2. GitHub webhook delivery log — show event payload dispatched
3. FastAPI backend receiving the webhook
4. SecurePulse Repositories screen refreshing with updated data

---

V-03 — CRITICAL: AI Copilot Security Investigation

File: `media/videos/demo_03_ai_copilot_cve.mp4`

Steps to record:
1. Open AI Copilot tab from bottom navigation
2. Type: How do I fix CVE-2024-3094 in XZ?
3. Response animates in with remediation plan, Dockerfile patch, Semgrep rule
4. Second query: How do I handle a SQL injection finding? — show alternate response

---

V-04 — CRITICAL: Live Dashboard Mode Switch

File: `media/videos/demo_04_live_mode_switch.mp4`

Steps to record:
1. App in Demo Mode — dashboard shows 88% posture, demo data
2. Navigate to Settings → switch to Live Mode → configure Render Cloud URL
3. Tap Test Connection — health ping returns OK with latency
4. Navigate back to Dashboard — live data loads from cloud
5. Navigate to Alerts — live feed from Render

V-05 — CRITICAL: Complete Security Incident Lifecycle

File: `media/videos/demo_06_e2e_lifecycle.mp4`

Steps to record:
1. Detection: Wazuh triggers Brute Force alert → appears in SecurePulse feed
2. Triage: open alert detail → read syslog, origin IP, severity
3. Investigation: switch to AI Copilot → ask for remediation advice
4. Action: return to alert → tap Acknowledge → status changes to Investigating
5. Report: open Reports screen → generate SOC 2 PDF → share via Android share sheet

This is the complete SOC analyst workflow from detection to documented resolution.

V-06 — HIGH: WebSocket Live Streaming

File: `media/videos/demo_05_websocket_live.mp4`

Steps to record:
1. Android Studio logcat open — filter [WebSocketService]
2. App connects → logcat shows Status: connected and /ws/alerts URL
3. Trigger an event on backend — new alert arrives via WebSocket
4. Show live update in app without manual page refresh

V-07 — HIGH: Semgrep SAST Scan Trigger

File: `media/videos/demo_08_semgrep_scan.mp4`

Steps to record:
1. Navigate to Repositories → select a repository
2. Tap Trigger Scan button
3. Scan queued → findings load
4. Navigate to Findings screen — show critical findings with CWE references

---

V-08 — MEDIUM: Android APK Installation from Release Build

File: `media/videos/demo_09_apk_install.mp4`

Steps to record:
1. Terminal: flutter build apk --release completing successfully
2. adb install securepulse-release.apk on physical device
3. App icon appears on Android home screen
4. First launch — biometric prompt appears

Part 5 — Capture Priority Checklist

ID	Document	Section	Type	Filename	Priority	Done
M-006	PROJECT_REPORT	Ch. 4.1	DIAGRAM	fig_4_1_overall_architecture.png	CRITICAL	PENDING
M-007	PROJECT_REPORT	Ch. 4.2	DIAGRAM	fig_4_2_mobile_clean_architecture.png	CRITICAL	PENDING
M-011	PROJECT_REPORT	Ch. 4.6	DIAGRAM	fig_4_5_websocket_fallback_flow.png	CRITICAL	PENDING
M-034	PROJECT_REPORT	Ch. 8	SCREENSHOT	fig_8_1_flutter_test_15_passed.png	CRITICAL	PENDING
M-035	PROJECT_REPORT	Ch. 8.7	SCREENSHOT	fig_8_2_flutter_analyze_clean.png	CRITICAL	PENDING
M-038	PROJECT_REPORT	Ch. 9	SCREENSHOT	fig_9_1_render_dashboard.png	CRITICAL	PENDING
M-039	PROJECT_REPORT	Ch. 9	SCREENSHOT	fig_9_2_api_health_live.png	CRITICAL	PENDING
M-041	PROJECT_REPORT	Ch. 9.9	SCREENSHOT	fig_9_4_android_release_build.png	CRITICAL	PENDING
M-042	PROJECT_REPORT	Ch. 9.9	SCREENSHOT	fig_9_4b_apk_artifact.png	CRITICAL	PENDING
M-045	PROJECT_REPORT	Ch. 6.22	VIDEO	demo_01_launch_dashboard.mp4	CRITICAL	PENDING
M-046	PROJECT_REPORT	Ch. 6.22	VIDEO	demo_02_wazuh_realtime_alert.mp4	CRITICAL	PENDING
M-047	PROJECT_REPORT	Ch. 6.22	VIDEO	demo_03_ai_copilot_cve.mp4	CRITICAL	PENDING
M-048	PROJECT_REPORT	Ch. 9.14	VIDEO	demo_04_live_mode_switch.mp4	CRITICAL	PENDING
M-016	PROJECT_REPORT	Ch. 6.2	SCREENSHOT	fig_6_1_splash_shimmer_screen.png	CRITICAL	PENDING
M-017	PROJECT_REPORT	Ch. 6.5	SCREENSHOT	fig_6_2_login_screen.png	CRITICAL	PENDING
M-018	PROJECT_REPORT	Ch. 6.5	SCREENSHOT	fig_6_3_biometric_challenge.png	CRITICAL	PENDING
M-019	PROJECT_REPORT	Ch. 6.8	SCREENSHOT	fig_6_4_repositories_list.png	CRITICAL	PENDING
M-021	PROJECT_REPORT	Ch. 6.14	SCREENSHOT	fig_6_5_ai_copilot_cve_response.png	CRITICAL	PENDING
M-022	PROJECT_REPORT	Ch. 6.15	SCREENSHOT	fig_6_6_executive_dashboard.png	CRITICAL	PENDING
M-023	PROJECT_REPORT	Ch. 6.16	SCREENSHOT	fig_6_7_soc_alerts_feed.png	CRITICAL	PENDING
M-050	WALKTHROUGH	Sec 9	SCREENSHOT	fig_w_02_health_check_response.png	CRITICAL	PENDING
M-051	WALKTHROUGH	Sec 10	SCREENSHOT	fig_w_03_login_screen_filled.png	CRITICAL	PENDING
M-052	WALKTHROUGH	Sec 12	SCREENSHOT	fig_w_04_avd_emulator_running.png	CRITICAL	PENDING
M-053	WALKTHROUGH	Sec 14	SCREENSHOT	fig_w_05_dashboard_dark.png	CRITICAL	PENDING
M-055	WALKTHROUGH	Sec 15	SCREENSHOT	fig_w_07_repositories_list.png	CRITICAL	PENDING
M-057	WALKTHROUGH	Sec 16	SCREENSHOT	fig_w_09_alerts_feed.png	CRITICAL	PENDING
M-059	WALKTHROUGH	Sec 17	SCREENSHOT	fig_w_11_ai_copilot_sql_chat.png	CRITICAL	PENDING
M-063	WALKTHROUGH	Sec 21	VIDEO	demo_05_websocket_live.mp4	CRITICAL	PENDING
M-067	WALKTHROUGH	Sec 25	SCREENSHOT	fig_w_18_render_service_live.png	CRITICAL	PENDING
M-068	WALKTHROUGH	Sec 30	VIDEO	demo_06_e2e_lifecycle.mp4	CRITICAL	PENDING

Part 6 — Suggested media/ Directory Structure

```
securepulse-mobile/
└── media/
    ├── screenshots/
    │   ├── app/
    │   │   ├── fig_6_1_splash_shimmer_screen.png
    │   │   ├── fig_6_2_login_screen.png
    │   │   ├── fig_6_3_biometric_challenge.png
    │   │   ├── fig_6_6_executive_dashboard.png
    │   │   ├── fig_6_4_repositories_list.png
    │   │   ├── fig_6_7_soc_alerts_feed.png
    │   │   ├── fig_6_5_ai_copilot_cve_response.png
    │   │   └── fig_6_8_settings_environment.png
    │   ├── backend/
    │   │   ├── fig_9_1_render_dashboard.png
    │   │   ├── fig_9_2_api_health_live.png
    │   │   └── fig_9_3_render_env_vars.png
    │   ├── testing/
    │   │   ├── fig_8_1_flutter_test_15_passed.png
    │   │   └── fig_8_2_flutter_analyze_clean.png
    │   ├── deployment/
    │   │   ├── fig_9_4_android_release_build.png
    │   │   └── fig_9_4b_apk_artifact.png
    │   └── integrations/
    │       ├── fig_w_15_wazuh_agent_alerts.png
    │       ├── fig_w_16_github_webhook_delivery.png
    │       └── fig_w_17_semgrep_findings_mobile.png
    ├── diagrams/
    │   ├── fig_4_1_overall_architecture.png
    │   ├── fig_4_2_mobile_clean_architecture.png
    │   ├── fig_4_3_backend_architecture.png
    │   ├── fig_4_5_websocket_fallback_flow.png
    │   └── fig_4_6_ai_copilot_flow.png
    └── videos/
        ├── demo_01_launch_dashboard.mp4
        ├── demo_02_wazuh_realtime_alert.mp4
        ├── demo_03_ai_copilot_cve.mp4
        ├── demo_04_live_mode_switch.mp4
        ├── demo_05_websocket_live.mp4
        ├── demo_06_e2e_lifecycle.mp4
        ├── demo_07_github_webhook.mp4
        ├── demo_08_semgrep_scan.mp4
        └── demo_09_apk_install.mp4
```

Notes

No media has been generated, inserted, or invented. Every item in this audit is PENDING capture.

The fastest path to a near-complete document is: run flutter test, run flutter analyze, then take one screenshot session of the app on the Android emulator in Demo Mode. That single session covers 14 of the 30 CRITICAL items and eliminates the most glaring evidence gaps in PROJECT_REPORT.md and WALKTHROUGH.md.

Diagrams M-006 through M-008 and M-011 (Figures 4.1-4.5, cited in the List of Figures) are the highest-leverage items. Tools such as draw.io, Mermaid Live, or Excalidraw can export PNG diagrams — no running deployment needed.

To insert media once captured, use standard Markdown image syntax in the source .md files:
 Figure 8.1 — flutter test 15/15 result